Service Package Recommendation for Mashup Creation via Mashup Textual Description Mining

Qi Gu[1,2], Jian Cao[1,*], Qianyang Peng[1]
1. Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China
2. School of Computer Science and Technology, Nantong University, Nantong, China
Email: guqi@sjtu.edu.cn, cao-jian@sjtu.edu.cn, az950512@sjtu.edu.cn

***Abstract*—Mashup is a developer-centric technique which allows developers to compose existing Web services together to create innovative or consolidated web applications. However, the rapid growth in the number of services and the myriad of functionally similar services make it difficult for developers to select appropriate ones to develop new applications. Therefore, it is vital to recommend a set of suitable services for mashup creation based on functionalities of services and their relationships. To this end, we propose a service package recommendation approach for mashup development, which is based on mashup textual description mining to discover semantic relationships among services. Specifically, discourse analysis of computational linguistics is utilized to uncover the structures underneath mashups' functional specifications, then the semantic relationships between services can be learned from their appearances and the constructed structures in mashup specifications. Accordingly, we are able to recommend a package of services that can be used together with high compatibility for a new mashup to be developed. We evaluate our approach on a real-world dataset. Experimental results show that our approach achieves higher accuracy and outperforms other comparative ones.**

Keywords-component; service recommendation; mashup creation; discourse analysis

I. INTRODUCTION

With the development of Web 2.0, Web services gain a strong momentum of development in ubiquitous environments. As single service may not satisfy users' needs directly, Mashup has become a significant technology in the field of services-oriented computing. Mashup is a novel technique for combining existing services together to create a new Web application [1]. For example, a developer could combine Foursquare and GeoNames with Google Maps to create a travel mashup: *Travel the World*, which provides travel guides when you travel to a country.

Mashup can satisfy users' complex requirements so that it becomes popular among all the service composition implementations [2]. Developers, even nonprogrammers can create interesting mashups by "drag and drop" actions even within a web browser. In recent years, a number of mashup platforms have been developed by various industry vendors, such as Yahoo! Pipes, IBM DAMIA and WSO2 Mashup Server. They provide an interface for building mashups that aggregate data and services from different sources. Thus, the development cycle and costs associated with a mashup can be dramatically reduced.

With the increasing number of services that have similar functions, it has been an urgent problem to recommend appropriate services for creating a mashup efficiently. However, how to recommend appropriate services to mashup developers remains a daunting task. In the past few years, service recommendation has received many interests and most approaches are limited to proposing substitutable services with similar functionality. However, in the procedure of mashup creation, developers need a set of services with complementary functions rather than a set of services with similar functions. We call the problem of recommending a set of compatible services for mashup development as "*Service Package Recommendation*". Not surprisingly, there have been some service recommendations proposed for mashup which are mainly based on the frequent pattern mining model, which only reveals the co-occurrences of services in a mashup. However, there are complex semantic relationships among services, which should be considered during mashup creation. Unfortunately, current approaches have not incorporated the semantic relations among services into the recommendation model in an effective way. The reason may be due to it is a hard to learn semantic relationships among services from real applications.

Existing mashups usually accompany with description documents in natural language. It is possible to analyze these existing mashups together with their descriptions to discover latent semantic relations among services, which can be applied to service package recommendation for mashup development. Specifically, we use discourse analysis to segment the textual description into sentence blocks and generalize the specific associations between sentence blocks into semantic relationships between services. As a result, we come up with a novel service package recommendation approach via mashup textual description mining.

The main contributions of this paper are as follows:

- We propose a novel approach to uncover semantic relations between services by analyzing and mining existing mashups and their descriptions using discourse analysis, which serves as the basis for service package recommendation.

* Jian Cao is the corresponding author

978-1-5090-2675-3/16 $31.00 © 2016 IEEE
DOI 10.1109/ICWS.2016.65

- We design a service package recommendation approach, which takes the functional specification of a mashup to be developed as the only input.
- We carried out experiments on a real-world dataset, which shows that our approach is effective.

The remainder of this paper is organized as follows. Section II introduces related works. Section III presents a motivating use case and formalizes the problem. Section IV describes our proposed approach. Section V presents the evaluation methodology and results. Finally, we conclude the paper with final comments in Section VI.

II. Related Work

Service recommendation has attracted a lot of attentions. In this section, we give an overview of the recent works on service recommendation and service recommendation for mashup development.

A. Service Discovery and Recommendation

Many service discovery and recommendation methods have been proposed. Some of them are based on the traditional service descriptions and more of them are based on semantic descriptions with technologies such as RDF, OWL-S and WSDL-S. Ontologies have been identified as the basis for semantic annotations [3]. Nevertheless, it is difficult to annotate elements in web services, which may involve a huge amount of human efforts [4]. Therefore, semantic technologies are still not widely used in industry [5]. Thus, the most practical service discovery and recommendation approaches are based on the functional specifications of services which are usually described in natural language.

Among various service recommendation approaches based on a given functional specification, traditional information retrieval methods are intuitive ones. Currently, with the assistances of some web service corpuses like *ProgrammableWeb*, technologies like vector space model and inverse document frequency are used to build service search engines, which can find services based on syntactic matching degrees between the functional specification and the services' descriptions. For instance, Platzer et al. [6] use the vector space model to improve the performance of search engines for services. Torres et al. [7] rank the APIs according to the tradeoff between the cosine distance and APIs' popularities. Furthermore, in recent years, sophisticated text mining approaches are adopted in service recommendation, for instance, Aznag et al. [4] extract topics from service descriptions and discover services based on constructed concept lattices.

B. Service Recommendation for Mashup Development

Mashup development is an End User Development (EUD) application approach with promising future [8]. Thus, service recommendation for mashup development has attracted the attentions from many researchers.

1) Semantic-based Approaches

Semantic-based approaches compute the semantic similarities between a functional specification and services' descriptions. Then the services with the highest matching degree are recommended. Elmeleegy et al. [9] describe a recommendation tool named MashupAdvisor, which uses a semantic matching algorithm and a metric planner to generate the mashup. In recent years, topic model and vector space model are also adopted. For instance, Li et al. [10] utilize the relational topic model to recommend services for mashup development.

2) Frequent Pattern-based Approaches

Since services exist in the form of combinations in mashups, frequent pattern mining has become a popular solution in service recommendation for mashup development. It is based on the assumption that when two services have been put together in many existing mashups, they are very likely to appear together in the future. Oliveira et al. [11] propose a service recommendation approach for mashup development via frequent pair mining. Furthermore, Zhang et al. [12] present a recommender system to mine longer frequent sequences. Goarany et al. [13] apply association rule mining techniques to discover relationships between APIs and mashups. Chowdhury et al. [14] propose an approach to mine reusable composition patterns and recommend services via pattern matching and retrieval. Although co-occurrence information among services are useful for recommendation, the reasons behind the co-occurrences, i.e., the semantic relationships among services are not revealed and made use of, which largely reduce the recommendation effects.

3) Social Relationship-based Approaches

Some researchers are also investigating the possibility of applying recommender systems and approaches based on social relationships, which consist of comprehensive relationships among services, service providers and service consumers [2][15][16][20]. This approach relies on more comprehensive information collected from services, service providers and service consumers.

Our recommendation approach only relies on historical mashups and their descriptions. Specifically, it applies discourse analysis of computational linguistics to discover semantic relationships among services, which provides a new way for service analysis and recommendation.

III. Motivation and Problem Definition

In this section, we present a case study to illustrate how semantic relationships between services can enhance the process of service recommendation for mashup creation. Also, we formalize the problem.

A. A Motivating Use Case

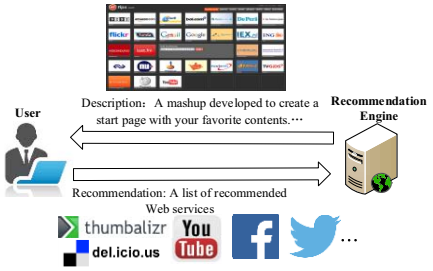


Figure 1. Service Recommendation Use Case

Figure 1 shows a service recommendation use case for mashup creation. When a user is going to create a new mashup application that can produce a start page with favorite contents, the user first describes the requirements as "A mashup developed to create a start page with your favorite contents. Import your photos, videos, music, bookmarks. View your favorite websites in one place. Discover similar contents. Collect, save and share your contents."

The recommendation engine firstly decomposes the mashup description into several sentence blocks. Using *the discourse parser tool*, the engine will also identify relationships between these sentence blocks. Then each sentence block will be matched with several services from the repository. Since semantic relationships among services in the repository have been learned from historical mashups and their descriptions, the recommendation engine will try to recommend a set of services whose relationships are consistent with the relationships among sentence blocks.

In this scenario, the recommendation engine will recommend *Flickr, del.icio.us, YouTube, Last.fm, Facebook, Twitter, LinkedIn, Thumbalizr* and *Hyves* to build the mashup application which presents the start page with favorite websites.

B. Problem Definition

Given the textual description D_m of a mashup m to be created, we parse the textual description D_m into statement blocks $S^{(m)} = \{s_1^{(m)}, s_2^{(m)}, \dots, s_{|S^{(m)}|}^{(m)}\}$, where $|S^{(m)}|$ is the number of statement blocks that the description decomposed into. Each statement block $s_i^{(m)}$ is related to a set of services $WS(s_i^{(m)}) = \{ws_i^1, ws_i^2, \dots, ws_i^{|WS(s_i^{(m)})|}\}$, where $|WS(s_i^{(m)})|$ is the number of services recommended to statement block $s_i^{(m)}$.

With the relationships exploited from the historical data, the service package $SP = \{ws_1, ws_2, \dots, ws_{|SP|}\}$ is recommended which can fulfil the user's need by maximizing some utility function, where $|SP|$ is the number of services in the service package.

IV. Framework and Algorithms

In this section, first, we present the framework of our proposed service package recommendation approach. Then we introduce discourse parser, which is a natural language tool to analyze the textual description of a given mashup. Finally, we describe the algorithms for service recommendation and service package recommendation for mashup development in detail.

A. Service Package Recommendation Framework

Figure 2 shows the framework for service package recommendation. It mainly consists of two parts:

1) Service Recommendation Engine

It receives the textual description of a mashup, which depicts the functional specification of the desired application. This specification is often written in natural language. A segmenter breaks the description into EDUs, which are the ***E***lementary ***D***iscourse ***U***nits in discourse analysis. Based on the EDUs, discourse parser links the EDUs into a labeled tree (Discourse Tree). Also, we use WordNet [17] to match EDUs with services' descriptions, which forms a large lexical database of English. Thus, we can recommend a list of services to each EDUs.

2) Service Repository

A service repository with enough historical mashup records is the knowledge base of our approach. We segment the description of each mashup into EDUs and construct a corresponding discourse tree. Based on the discourse analysis and LSI (Latent Semantic Indexing) model, we can generate the relation matrix of services.

B. Discourse Analysis

The functional description of a mashup is not merely a sequence of independent and isolated sentences, in contrast, the meaning of a sentence also relates to the adjacent sentences. The description exhibits a finer-level structure

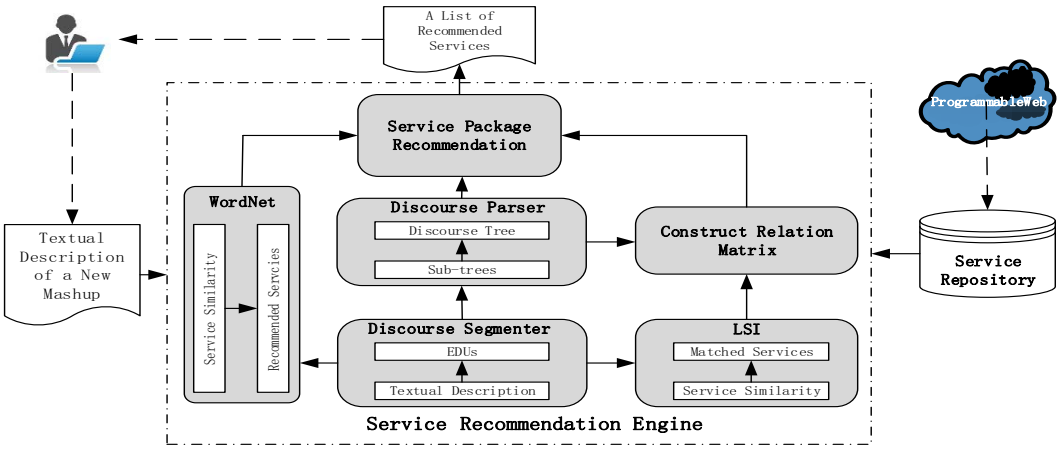


Figure 2. The Overall Structure of Our Framework

called coherence structure in which the clauses and sentences are logically bund together to express the meaning as a whole. Discourse analysis is well-studied in many natural language processing applications, which seeks to uncover the coherence structure between the related sentences.

In order to explore the coherence structure of the functional descriptions of a mashup so as to uncover the relationships between services, we utilize CODRA [18], which is a complete probabilistic discriminative framework for discourse analysis. CODRA uncovers the coherence structure under the text and posits a tree representation of a discourse. Thus, it represents the description by labeled hierarchical structures called Discourse Trees (DTs). Figure 3 shows the Discourse Tree for the use case mentioned in Section III.

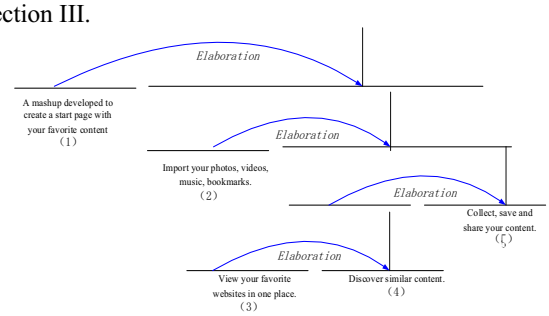


Figure 3. The Discourse Tree for the Use Case

The leaves of the DT correspond to the EDUs in the description. Horizontal lines indicate statement blocks. Adjacent EDUs are identified with coherence relations (e.g. Elaboration), they are connected by curved arrows with the identified relationships. CODRA comprises two parts: discourse segmenter and discourse parser.

1) Discourse Segmenter

Discourse segmenter breaks the functional description of a mashup into a sequence of EDUs. Actually, the task of discourse segmenter is to find EDU boundaries inside sentences. CODRA utilizes a maximum entropy model for binary classification. Currently, automatic discourse segmentation has attained accuracies close to human performances [19].

2) Discourse Parser

Discourse parser links the EDUs into DT. In the process of building the DT, discourse parser determines which discourse units to relate, and what relations to use when the EDUs are identified. Discourse parser comprises two probabilistic discriminative parsing models: intra-sentential parser and multi-sentential parser. Intra-sentential parser produces one or more discourse sub-trees for each sentence while multi-sentential parser generates a full discourse tree for the document from these sub-trees.

C. Construct Relation Matrix

Discourse analysis breaks the textual description into EDUs and constructs a discourse tree. In order to match each EDU to a service and learn the relations between services, we use LSI method, which tries to overcome the problem of lexical matching by using statistically derived conceptual indices instead of individual words for retrieval. In order to depict the correlations between keywords, we utilize the descriptions of all the mashups and services in our real-world dataset to construct the service repository. Then, we clear the data through word segmentation, removing the stop words, and stemming.

By utilizing the LSI model, we compute the similarity between EDUs and services. The service with the highest similarity value is the matched one. In order to simplify the mapping process, if the service name appears in the EDU we regard it as the matched service directly.

Based on the constructed discourse tree, we can then obtain semantic relations between services by inheriting the relationships between corresponding EDUs. At the same time, the co-occurrence relationships can be easily obtained from historical mashups:

a) Co-occurrence Relation

If two services are used by the same mashup, we say they have co-occurrence relationship. For example, mashup m is composed of service A, service B, service C, we say service A, B and C have a co-occurrence relationship. Co-occurrence is the elementary relation between services.

b) Semantic Relation

Based on the elementary co-occurrence relation between services, we discover several different semantic relations between services using the tool CODRA. Table I shows different relations discovered in our real-world dataset in detail. E.g. “sameblock” means two services belong to the same block, “count=1000” means the number of different service pairs have this semantic relation comes to 1000, whereas “sum=4330” means the total number of this semantic relation comes to 4330.

TABLE I. STATISTICS OF RELATIONS BETWEEN SERVICES IN OUR DATA SET

Name	Count	Sum
sameblock	1000	4330
comparison-comparison	4	4
topic-comment-topic-comment	2	2
temporal-temporal	46	46
span-condition	18	18
condition-span	18	18
attribution-span	142	154
span-attribution	142	154
enablement-span	254	267
span-enablement	254	267
manner-means-span	54	54
span-manner-means	54	54
span-contrast	13	13
contrast-span	13	13
joint-joint	1000	1274
background-span	107	109
span-background	107	109
span-cause	45	45
cause-span	45	45
span-elaboration	1000	1566
elaboration-span	1000	1566
same-unit-same-unit	690	752
span-temporal	16	16
temporal-span	16	16
explanation-span	17	17

span-explanation	17	17
topic-comment-span	2	2
span-topic-comment	2	2
contrast-contrast	124	126
comparison-span	1	1
span-comparison	1	1
together	1000	9990

Most of the relations shown in Table I are symmetric, for example, "*span-condition*" and "*condition-span*". Specifically, the co-occurrence relationship is represented as "*together*", which makes our model also considers the co-occurrence relationship as frequent pattern-based approach does.

With the discovered relations between services, we construct service relation matrix M_{rel}, which depicts the semantic relations between services.

D. Service Recommendation

For the purpose of measuring the importance of words in the document, we use TF-IDF (term frequency-inverse document frequency) as a numeric statistic to weight how importance a word is to a document. We extract keywords from service descriptions and sentence blocks according to TF-IDF. We set the threshold value to 0.15, which is appropriate according to our experiments. We select a word as a keyword if and only if it's TF-IDF weight is over this threshold.

The lexical matching considers the similarity between words, and may miss some information. For example, *Cat* and *Animal* has low similarity according to word matching, but they are subordination. As lexical matching methods can be inaccurate when they are used to match documents, after picking up keywords in the service descriptions and sentence blocks in mashup description, we use WordNet to compute the semantic similarity between keywords. The similarity definition between a service ws_i and a sentence block $s_j^{(m)}$ is denoted as (1):

$$Sim\left(ws_i, s_j^{(m)}\right) = WordNet\left(ws_i, s_j^{(m)}\right) * \alpha + pop(ws_i) * (1-\alpha), \qquad (1)$$

where $WordNet(ws_i, s_j^{(m)})$ is the semantic similarity between service ws_i and sentence block $s_j^{(m)}$, $pop(ws_i)$ is the popularity of service ws_i, α is a control parameter which indicates the tradeoff between sematic similarity and popularity. We use the number of followers in the web site to indicate the popularity of a service because we believe the service that has more followers may have a higher popularity. Thus, we can recommend a list of services for each sentence block according to the similarity score which considers both semantic similarity and popularity. Services with high similarity score (over a given threshold t) are listed in the recommendation list.

E. Servcie Package Recommendation

A service package can be recommended by taking out a service from each service list corresponding to each sentence block so that the semantic relationships among selected services are more consistent with the relationships among sentence blocks comparing with other packages. This procedure can be described as the following algorithm.

Algorithm 1: Service Package Recommendation

Input: the functional specification D_m of the mashup m;
the service relation matrix M_{rel};
similarity threshold t;
parameter k, z, r;
Output: the recommended service package SP;

```
generate EDUs S^(m) of D_m;
SP ={}; WS ={}; RS ={}
for each EDU s_j^(m) in S^(m) do
    for each Web service ws_i in service repository do
        calculate Sim(ws_i, s_j^(m)) according to Eq.(1)
        if Sim > t then add ws_i to WS
    end for
    select the foremost k Web services in WS to RS;
    select the Web service with Sim >0.99 in RS to SP;
    select the foremost z Web services in RS to SP;
end for
for each ws_i in RS do
    for each ws_j in RS do
        find relationship rel between ws_i and ws_j;
        if M_rel[ws_i, ws_j]> r then add them to SP;
    end for
end for
return a service package SP;
```

V. EXPERIMENTS

In this section, we first introduce the experiment settings and then describe a set of experiments to evaluate the performances of our proposed service package recommendation approach for mashup development. The experiments focus on comparing the performances of our proposed approach with the mere keyword based approach and semantic-based approach, studying the impacts of proposed relationships between services, and the impacts of different parameters' settings in our approach.

A. Experiment Settings

1) Dataset

The dataset comes from ProgrammableWeb.com, which is the largest service information sharing web site. It collects information not only of services but also of mashups. We crawled 10,240 services as well as 7674 mashups from the ProgrammableWeb. In this dataset, each service contains the information such as name, descriptions, followers, developers, tags and category. Table II shows some statistics of this real-world dataset.

TABLE II. STATISTICS OF PROGRAMMABLEWEB DATA FOR EVALUATION

Data Type	Statistic
number of services(short for #services)	10240
number of mashups(short for #mashups)	7674
average number of services per mashup	2.06
services used in at least one mashup	1347
number of mashups with more than one service	2601

As shown in Table II，we can discover that the usage frequencies of services in mashups are imbalance and sparse. We use Gephi0.8.2 (https://gephi.org) to depict the correlations among services. If two services are connected with a line, they co-exist in the same mashup. With the increase of the number of its connected lines, we also increase the diameter of the node. As shown in Figure 4, we can find that Google Maps has most co-occurrences with other services; Twitter, YouTube and Flicker take the second place.

Figure 4. Statistic on Correlation of APIs in Mashup

2) Comparative Methods

WVSM and WJaccard are implemented as the comparative methods, which are all keyword-based approaches [10].

*a) **WVSM(**Weighted **V**ector **S**pace **M**odel)*

A mashup m is represented as a binary vector $v^{(m)}$ while a service ws is represented as a binary vector $v^{(ws)}$, with one component for each dictionary term. The distance between mashup and service can be found by computing the Cosine Similarity between their vector representations. The popularity of the service is utilized as a weight. Then WVSM ranks services according to the product of similarity and weight. That is:

$$p_{wvsm}(m, ws) = pop(ws)\frac{v^{(m)}v^{(ws)}}{||v^{(m)}||||v^{(ws)}||}. \quad (2)$$

*b) **WJaccard(**Weighted **Jaccard)***

Jaccard similarity coefficient is defined as the number of common terms shared by mashup m and service ws divided by the number of all unique terms contained by either mashup m or service ws. The popularity of the service is utilized as a weight as well. Then WJaccard ranks services according to the product of similarity and weight. That is:

$$p_{wvsm}(m, ws) = pop(ws)\frac{|w^{(m)} \cap w^{(ws)}|}{|w^{(m)} \cup w^{(ws)}|}. \quad (3)$$

3) Evaluation Metrics

We evaluate our approach with precision and recall. They are the most common metrics to evaluate a recommendation.

$$precision = \frac{tp}{tp+fp}. \quad (4)$$

$$recall = \frac{tp}{tp+fn}. \quad (5)$$

In our experiments, tp (True Positive) refers to the number of correct recommended services, which are actually used in the mashup. fp (False Positive) refers to the number of incorrect recommended services, which are not used in the mashup. fn (False Negative) indicates the number of services not being recommended, which are really used in the mashup. Precision measures the proportion of recommended services that are truly used in the mashup, while recall measures the proportion of used services that are correctly recommended. Recall also measures the coverage of used services in recommendations.

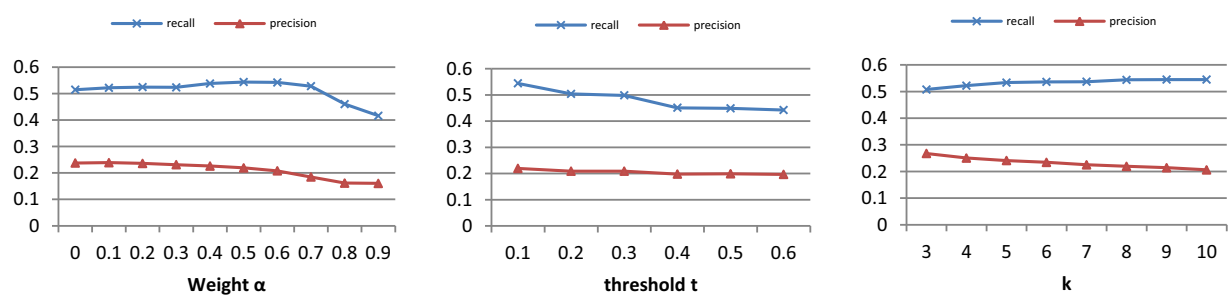
B. Model Training

Of all crawled mashups, 90% are selected randomly and applied to train the model and the remaining 10 percent are used as a testing set. There are many parameters in our model. In this section, we show how they are fitted.

In order to choose the befitting parameters for our proposed approach, extensive experiments have been done, where weight value α changes from 0 to 0.9, threshold value t changes from 0.1 to 0.5, the number of services recommended to each EDU value k changes from 3 to 10. Figure 5 shows the results upon different parameter values.

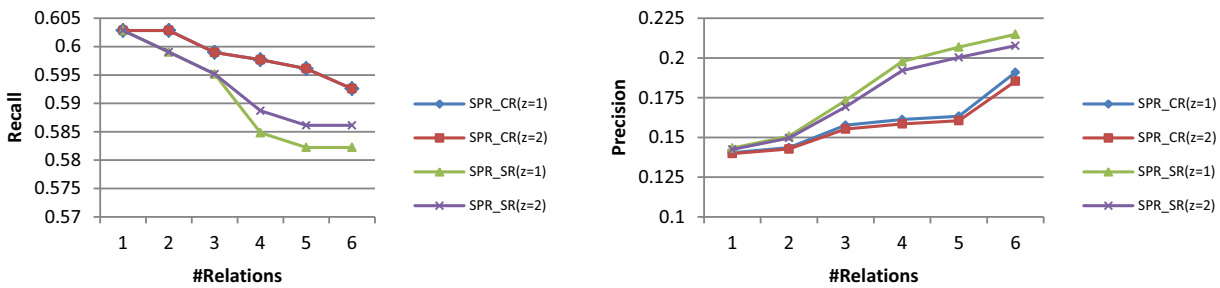
1) Impact of α

To evaluate the impact of weight in similarity computation, we vary the weight value from 0 to 0.9 with a step size of 0.1. Figure 5(a) shows that the precision reaches the highest value when weight is 0.1, while recall reaches the maximum value when weight is 0.5. Our target is to recommend as more services as users may need. Consequently, we prefer a high recall to ensure a larger



(a). impact of α (b). impact of t (c). impact of k

Figure 5. Impact of Different Parameters



(a) Recall (b) Precision

Figure 6. Results of Service Package Recommendation Approaches over Various #Relation and #z

candidate set for service recommendation. Thus, the value of weight is set to 0.5 to compute the similarity in our comparison experiments.

2) Impact of t

To decide the most appropriate similarity threshold t, we study the impact of t by varying the value of this parameter from 0.1 to 0.5 with a step size of 0.1. Figure 5(b) shows that both the precision and recall decrease as threshold increases. This means our proposed approach achieves the best performance when threshold value is set to 0.1. The reason lies in that lower threshold may result in more services are contained in recommendations. Therefore, we take this value as the default setting in our comparative experiments.

3) Impact of k

Parameter k determines the number of services recommended to each EDU. We study the sensitivity of k by tuning the value of this parameter within the range of $\{3,4,\ldots,10\}$ with the step size of 1. As shown in Figure 5(c), with the increase of the number of recommended services, the recall remains high, however, the precision declines. Also, as the number becoming larger, the number of average recommended service increases. When the value reaches 8, the performance of our approach tends to be stable. Thus, we set k =8, which is a compromise among these values.

In conclusion, we set the value of weight α to 0.5, the value of similarity threshold t to 0.1, and the number of services recommended to each EDU k to 8.

C. Comparative Experiments and Analysis

We conducted comparative experiments using each of the methods described in this section:

(1) **SPR:** *Service Package Recommendation without Relation*;
(2) **SPR_CR:** *Service Package Recommendation with Co-occurrence Relation*;
(3) **SPR_SR**: *Service Package Recommendation with Semantic Relation;*

The results are shown in Figure 6 where various r (the number of relations which are used to characterize the correlations between services) are considered, as well as the number of z (the number of services which are the foremost ones according to the similarity between services and EDU).

As shown in Table III, the SPR gets the maximum recall, however, the average recommended service number rises up to 17. The precision of SPR is decreased to 10.87%. The recall of SPR-SR is close to that of the SPR; at the same time, the precision is considerably higher than others. The reason lies in that implicit semantic relations have been discovered from mashup records.

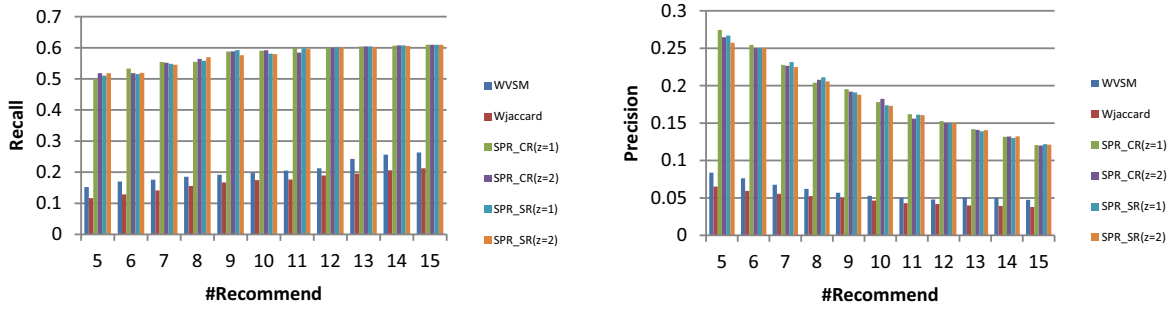
We also compared our approach with two other approaches: WVSM approach and WJaccard approach. Figure 7 shows the comparison results. We can conclude that our proposed approach is much better than the other two approaches.

VI. CONCLUSIONS

With the increasing number of services, automatic service recommendation places an important role in mashup development environments. In this paper, we focus on the approach of recommending a set of complementary services in the process of mashup development. We apply discourse analysis of computational linguistics as a new way for service analysis and recommendation. It is based on the fact

TABLE III APPROACH COMPARISON OVER PARAMETER K

	Recall					Precision					#Recommend(average)				
k	SPR	SPR_CR (z=1)	SPR_CR (z=2)	SPR_SR (z=1)	SPR_SR (z=2)	SPR	SPR_CR (z=1)	SPR_CR (z=2)	SPR_SR (z=1)	SPR_SR (z=2)	SPR	SPR_CR (z=1)	SPR_CR (z=2)	SPR_SR (z=1)	SPR_SR (z=2)
3	0.5194	0.5106	0.5184	0.4986	0.5184	0.2468	0.2671	0.2573	**0.2744**	0.2648	6.05	5.46	5.76	**5.15**	5.60
4	0.5483	0.5445	0.5445	0.5333	0.5419	0.2140	0.2406	0.2305	0.2545	0.2408	7.50	6.55	6.81	6.12	6.55
5	0.5585	0.5547	0.5547	0.5436	0.5521	0.1830	0.2244	0.2158	0.2419	0.2315	9.06	7.16	7.44	6.51	6.94
6	0.5798	0.5695	0.5695	0.5545	0.5592	0.1577	0.2129	0.2056	0.2350	0.2249	10.85	7.77	8.05	6.84	7.24
7	0.5847	0.5744	0.5744	0.5602	0.5641	0.1404	0.2002	0.1939	0.2223	0.2143	12.52	8.48	8.78	7.36	7.71
8	0.6029	0.5926	0.5926	0.5822	0.5861	0.1287	0.1910	0.1854	0.2149	0.2077	14.04	9.26	9.54	7.97	8.32
9	0.6080	0.5977	0.5977	0.5864	0.5903	0.1192	0.1818	0.1769	0.2059	0.1997	15.44	9.96	10.25	8.49	8.84
10	**0.6095**	0.5993	0.5993	0.5879	0.5918	0.1087	0.1696	0.1654	0.1953	0.1901	17.05	10.84	11.12	9.05	9.40



(b) Recall (b) Precision

Figure7. Comparison Results over Various #Recommended Services

that relationships between sentence blocks carry important information. In the learning stage, a discourse parser is applied to segment the textual descriptions of each mashup into sentence blocks, which are matched to services so that semantic relations between services can be learned. In the recommendation stage, our approach can recommend a service package by selecting a set of services that can realize the relationships among sentence blocks of a functional specification to the maximum extent. After preprocessing of the real-world data set crawled from the ProgrammableWeb, we performed experiments and experimental results verify the effectiveness of our approach.

ACKNOWLEDGMENT

This work is partially supported by China National Science Foundation (Granted Number 61272438,61472253), Research Funds of Science and Technology Commission of Shanghai Municipality (Granted Number 15411952502, 14511107702) and Cross Research Fund of Biomedical Engineering of Shanghai Jiao Tong University (YG2015MS61).

REFERENCES

[1] Benslimane, Djamal, Schahram Dustdar, and Amit Sheth. "Services mashups: The new generation of web applications." IEEE Internet Computing 5 (2008): 13-15.

[2] Xu W, Cao J, Hu L, et al. A social-aware service recommendation approach for mashup creation[C]//Web Services (ICWS), 2013 IEEE 20th International Conference on. IEEE, 2013: 107-114.

[3] Sivashanmugam K, Verma K, Sheth A P, et al. Adding semantics to web services standards[C]//Web Services (ICWS), 2003 IEEE 20th International Conference on. IEEE, 2003: 395-401.

[4] Aznag M, Quafafou M, Jarir Z. Leveraging formal concept analysis with topic correlation for service clustering and discovery[C]//Web Services (ICWS), 2014 IEEE International Conference on. IEEE, 2014: 153-160.

[5] Blake M B, Nowlan M F. A web service recommender system using enhanced syntactical matching[C]//Web Services, 2007. ICWS 2007. IEEE International Conference on. IEEE, 2007: 575-582.

[6] Platzer C, Dustdar S. A vector space search engine for web services[C]//Web Services, 2005. ECOWS 2005. Third IEEE European Conference on. IEEE, 2005: 9 pp.

[7] Torres R, Tapia B, Astudillo H. Improving Web API Discovery by leveraging social information[C]//Web Services (ICWS), 2011 IEEE International Conference on. IEEE, 2011: 744-745.

[8] Grammel L, Storey M A. A survey of mashup development environments[M]//The smart internet. Springer Berlin Heidelberg, 2010: 137-151

[9] Elmeleegy H, Ivan A, Akkiraju R, et al. Mashup advisor: A recommendation tool for mashup development[C]//Web Services, 2008. ICWS'08. IEEE International Conference on. IEEE, 2008: 337-344.

[10] Li C, Zhang R, Huai J, et al. A novel approach for API recommendation in mashup development[C]//Web Services (ICWS), 2014 IEEE International Conference on. IEEE, 2014: 289-296.

[11] De Oliveira F T, Murta L, Werner C, et al. Using provenance to improve workflow design[M]//Provenance and Annotation of Data and Processes. Springer Berlin Heidelberg, 2008: 136-143.

[12] Zhang J, Liu Q, Xu K. FlowRecommender: a workflow recommendation technique for process provenance[C]//Proceedings of the Eighth Australasian Data Mining Conference-Volume 101. Australian Computer Society, Inc., 2009: 55-61.

[13] Goarany K, Kulczycki G, Blake M B. Mining social tags to predict mashup patterns[C]//Proceedings of the 2nd international workshop on Search and mining user-generated contents. ACM, 2010: 71-78.

[14] Chowdhury S R, Daniel F, Casati F. Efficient, interactive recommendation of mashup composition knowledge[M]//Service-Oriented Computing. Springer Berlin Heidelberg, 2011: 374-388.

[15] Zhong Y, Fan Y, Huang K, et al. Time-aware service recommendation for mashup creation in an evolving service ecosystem[C]//Web Services (ICWS), 2014 IEEE International Conference on. IEEE, 2014: 25-32.

[16] Cao B, Liu J, Tang M, et al. Mashup service recommendation based on user interest and social network[C]//Web Services (ICWS), 2013 IEEE 20th International Conference on. IEEE, 2013: 99-106.

[17] Miller G A. WordNet: a lexical database for English[J]. Communications of the ACM, 1995, 38(11): 39-41.

[18] Joty S, Carenini G, Ng R T. CODRA: A novel discriminative framework for rhetorical analysis[J]. Computational Linguistics, 2015.41:385-435

[19] Fisher S, Roark B. The utility of parse-derived features for automatic discourse segmentation[C]//ANNUAL MEETING-ASSOCIATION FOR COMPUTATIONAL LINGUISTICS. 2007, 45(1): 488.

[20] Liu X, Fulia I. Incorporating User, Topic, and Service Related Latent Factors into Web Service Recommendation[C]//Web Services (ICWS), 2015 IEEE International Conference on. IEEE, 2015: 185-192.